

Computer Architecture Project Report

Design and Implementation of RISC-V Single Cycle Processor

Submitted By
Rajesh Joshi
Roll No: 23BEC155

Submitted To
Dr. Purvi Patel



Department of Electronics and Communication Engineering

Institute of Technology

Nirma University

Academic Year: 2025–2026

Contents

Abstract	2
1 Introduction	2
2 Literature Survey	2
3 Limitations of Existing Technologies	3
4 Proposed Methodology	3
5 RISC-V Instruction Formats	3
6 Control Signals	4
7 Processor Module Description	4
7.1 Program Counter	4
7.2 Instruction Memory	4
7.3 Register File	4
7.4 Control Unit	4
7.5 ALU	4
7.6 Data Memory	4
8 Instruction Execution Flow	4
9 Processor Block Diagram	5
10 Simulation Results	5
11 Analysis of Results	6
12 Conclusion	6
13 References	6
14 Appendix: Verilog Code	7

Abstract

RISC-V is a modern open-source instruction set architecture that allows researchers and engineers to design custom processors. In this project, a **Single Cycle RISC-V Processor** is implemented using Verilog Hardware Description Language.

A single cycle processor executes each instruction in one clock cycle. The processor performs instruction fetch, instruction decode, execution, memory access, and write-back operations in one cycle.

The architecture includes components such as Program Counter, Instruction Memory, Register File, Control Unit, ALU, Data Memory, Immediate Generator, and Multiplexers. Simulation results verify correct execution of instructions.

Keywords: RISC-V, Verilog HDL, Single Cycle Processor, Computer Architecture

1 Introduction

Processors are the main components of modern computing systems. They execute instructions stored in memory and perform calculations required by software applications.

RISC (Reduced Instruction Set Computer) architectures simplify processor design by using a small set of simple instructions. RISC-V is a modern open-source RISC architecture developed at the University of California, Berkeley.

Because RISC-V is open source, designers can implement their own processors without licensing restrictions. This makes it highly suitable for academic research and embedded system development.

In this project, a single cycle RISC-V processor is designed using Verilog HDL.

2 Literature Survey

Several processor architectures exist such as x86, ARM, MIPS, and RISC-V. Traditional processors often use complex instruction sets that require complicated hardware.

RISC architectures simplify instruction execution by using fixed instruction formats and load-store architecture.

Examples of RISC processors include:

- ARM Processors
- MIPS Processors
- RISC-V Processors

RISC-V provides flexibility and modular design which makes it ideal for modern processor development.

3 Limitations of Existing Technologies

Although modern processors are powerful, they have several limitations.

- Proprietary processor architectures require licensing fees.
- Complex instruction sets increase hardware complexity.
- High-performance processors consume more power.

Single cycle processors also have some drawbacks:

- All instructions take the same clock cycle.
- The clock cycle must be long enough for the slowest instruction.
- Hardware utilization is not efficient.

4 Proposed Methodology

The processor executes instructions using the following stages:

1. Instruction Fetch
2. Instruction Decode
3. Execute
4. Memory Access
5. Write Back

The architecture includes several modules such as program counter, register file, ALU, control unit, and memory units.

5 RISC-V Instruction Formats

Table 1: RISC-V Instruction Formats

Instruction Type	Fields
R-Type	funct7, rs2, rs1, funct3, rd, opcode
I-Type	imm[11:0], rs1, funct3, rd, opcode
S-Type	imm[11:5], rs2, rs1, funct3, imm[4:0], opcode
B-Type	imm, rs2, rs1, funct3, opcode

6 Control Signals

Table 2: Control Signals Generated by Control Unit

Opcode	Instruction	RegWrite	MemRead	MemWrite	Branch
0110011	R-Type	1	0	0	0
0000011	Load	1	1	0	0
0100011	Store	0	0	1	0
1100011	Branch	0	0	0	1

7 Processor Module Description

7.1 Program Counter

The program counter stores the address of the current instruction and updates it every clock cycle.

7.2 Instruction Memory

Instruction memory stores the program instructions and provides them based on the program counter value.

7.3 Register File

The register file contains 32 registers that store operands and intermediate results.

7.4 Control Unit

The control unit generates control signals required to operate the processor modules.

7.5 ALU

The Arithmetic Logic Unit performs operations such as addition, subtraction, AND, OR, and XOR.

7.6 Data Memory

Data memory is used for load and store instructions.

8 Instruction Execution Flow

The processor executes instructions in a sequence of operations:

Instruction Fetch → Instruction Decode → Execute → Memory Access → Write Back

This flow describes how the processor processes each instruction.

9 Processor Block Diagram

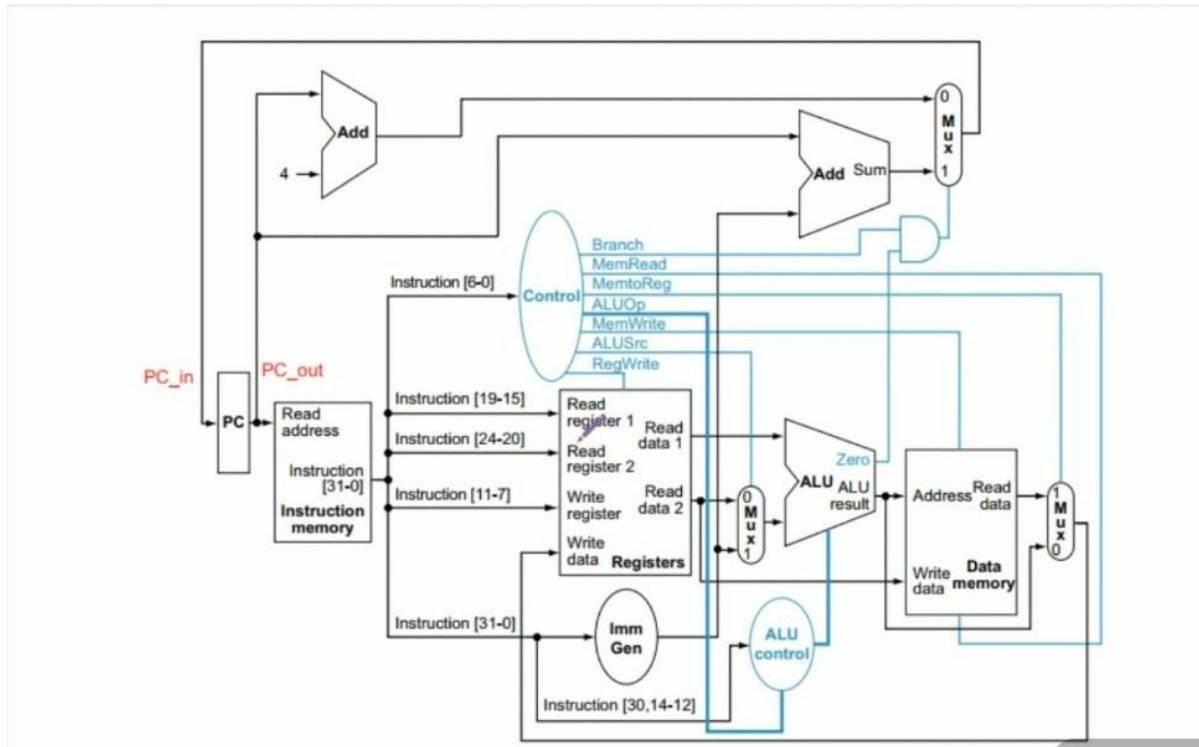


Figure 1: RISC-V Single Cycle Processor Architecture

10 Simulation Results

The processor design was simulated using a Verilog simulation tool. The waveform verifies correct processor operation including instruction fetch, ALU operation, register updates, and memory access.

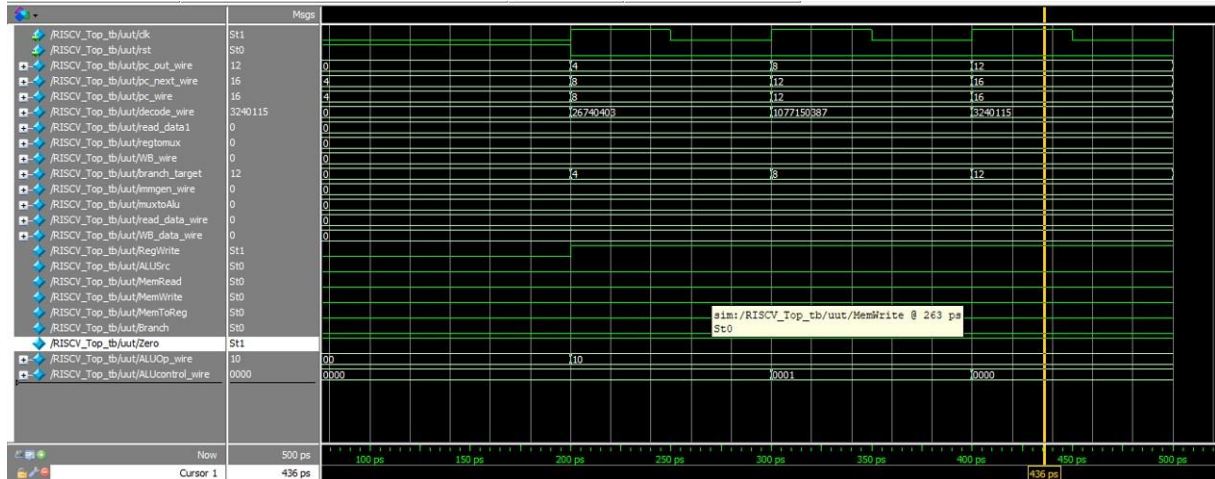


Figure 2: Simulation Waveform

11 Analysis of Results

Simulation results show correct processor behavior. The program counter increments correctly and instructions are fetched sequentially.

ALU operations are performed accurately and results are written back to registers. Memory operations for load and store instructions also function correctly.

12 Conclusion

This project successfully implements a RISC-V single cycle processor using Verilog HDL. The processor executes instructions correctly and demonstrates the working of fundamental CPU components.

The design provides a clear understanding of processor architecture and instruction execution. Future improvements may include implementing pipelining and supporting more RISC-V instructions.

13 References

1. Patterson, D. A., Hennessy, J. L., Computer Organization and Design RISC-V Edition.
2. RISC-V Instruction Set Manual.
3. M. Mano, Computer System Architecture.

14 Appendix: Verilog Code

```
1
2 module program_counter(input clk,input rst,input [31:0] pc_in,output
  reg [31:0] pc_out);
3 always @(posedge clk or posedge rst)
4 begin
5   if(rst)
6     pc_out <=32'b0;
7   else
8     pc_out <=pc_in;
9   end
10 endmodule
11
12
13 module pc_adder(input [31:0] pc_in,output [31:0] pc_next);
14 assign pc_next = pc_in + 4;
15 endmodule
16
17
18 module pc_mux(input [31:0] pc_in,input [31:0] pc_branch,input
  pc_select,output [31:0] pc_out);
19 assign pc_out = (pc_select) ? pc_branch : pc_in;
20 endmodule
21
22
23 module Instruction_Memory(input rst,input clk,input [31:0]
  read_address,output [31:0] instruction_out);
24
25 reg [31:0] I_Mem [0:255];
26 integer k;
27
28 assign instruction_out = I_Mem[read_address[9:2]];
29
30 always @(posedge clk or posedge rst) begin
31   if(rst)
32     begin
33       for(k=0;k<256;k=k+1)
34         I_Mem[k] = 32'b0;
35     end
36   else
```

```

37 begin
38 I_Mem[0]=32 ' b00000000000000000000000000000000 ;
39 I_Mem[1]=32 ' b0000000_11001_10000_000_01101_0110011 ;
40 I_Mem[2]=32 ' b0100000_00011_01000_000_00101_0110011 ;
41 I_Mem[3]=32 ' b0000000_00011_00010_111_00001_0110011 ;
42 I_Mem[4]=32 ' b0000000_00101_00011_110_00100_0110011 ;
43 I_Mem[5]=32 ' b0000000_00101_00011_100_00100_0110011 ;
44 I_Mem[6]=32 ' b0000000_00101_00011_001_00100_0110011 ;
45 I_Mem[7]=32 ' b0000000_00101_00011_101_00100_0110011 ;
46 end
47 end
48 endmodule
49
50
51 module Register_File(
52 input clk,
53 input rst,
54 input RegWrite ,
55 input [4:0] Rs1,
56 input [4:0] Rs2,
57 input [4:0] Rd,
58 input [31:0] Write_data ,
59 output [31:0] read_data1 ,
60 output [31:0] read_data2
61 );
62
63 reg [31:0] Registers [0:31];
64 integer k;
65
66 always @(posedge clk or posedge rst) begin
67 if(rst)
68 begin
69 for(k=0;k<32;k=k+1)
70 Registers[k]=32 'b0;
71 end
72 else if(RegWrite && Rd!=0)
73 Registers[Rd]=Write_data;
74 end
75
76 assign read_data1 = Registers[Rs1];
77 assign read_data2 = Registers[Rs2];

```

```

78
79 endmodule
80
81
82 module main_control_unit(
83 input [6:0] opcode,
84 output reg RegWrite,
85 output reg MemRead,
86 output reg MemWrite,
87 output reg MemToReg,
88 output reg ALUSrc,
89 output reg Branch,
90 output reg [1:0] ALUOp
91 );
92
93 always @(*) begin
94 case(opcode)
95
96 7'b0110011: {ALUSrc,MemToReg,RegWrite,MemRead,MemWrite,Branch,ALUOp}
          = {1'b0,1'b0,1'b1,1'b0,1'b0,1'b0,2'b10};
97
98 7'b0010011: {ALUSrc,MemToReg,RegWrite,MemRead,MemWrite,Branch,ALUOp}
          = {1'b1,1'b0,1'b1,1'b0,1'b0,1'b0,2'b10};
99
100 7'b0000011: {ALUSrc,MemToReg,RegWrite,MemRead,MemWrite,Branch,ALUOp}
          = {1'b1,1'b1,1'b1,1'b1,1'b0,1'b0,2'b00};
101
102 7'b0100011: {ALUSrc,MemToReg,RegWrite,MemRead,MemWrite,Branch,ALUOp}
          = {1'b1,1'b0,1'b0,1'b0,1'b1,1'b0,2'b00};
103
104 7'b1100011: {ALUSrc,MemToReg,RegWrite,MemRead,MemWrite,Branch,ALUOp}
          = {1'b0,1'b0,1'b0,1'b0,1'b0,1'b1,2'b11};
105 default: {ALUSrc,MemToReg,RegWrite,MemRead,MemWrite,Branch,ALUOp} =
          {1'b0,1'b0,1'b0,1'b0,1'b0,1'b0,2'b00};
106
107 endcase
108 end
109 endmodule
110
111

```

```

112 module immediate_generator(input [31:0] instruction,output reg
    [31:0] imm_out);
113
114 always @(*) begin
115     case( instruction [6:0])
116
117     7'b0010011: imm_out = {{20{instruction[31]}},instruction[31:20]};
118     7'b0000011: imm_out = {{20{instruction[31]}},instruction[31:20]};
119     7'b0100011: imm_out = {{20{instruction[31]}},instruction[31:25],
        instruction[11:7]};
120     7'b1100011: imm_out = {{19{instruction[31]}},instruction[31],
        instruction[7],instruction[30:25],instruction[11:8],1'b0};
121     default: imm_out = 32'b0;
122
123     endcase
124     end
125 endmodule
126
127 module ALU(
128     input [31:0] A,
129     input [31:0] B,
130     input [3:0] ALUcontrol_In,
131     output reg [31:0] Result,
132     output Zero
133 );
134
135 always @(*) begin
136     case( ALUcontrol_In)
137
138     4'b0000: Result = A + B;
139     4'b0001: Result = A - B;
140     4'b0010: Result = A & B;
141     4'b0011: Result = A | B;
142     4'b0100: Result = A ^ B;
143     default: Result = 32'b0;
144
145     endcase
146     end
147     assign Zero = (Result == 32'b0);
148
149 endmodule

```

```

150
151 module ALU_Control(
152 input [2:0] funct3,
153 input [6:0] funct7,
154 input [1:0] ALUOp,
155 output reg [3:0] ALUcontrol_Out
156 );
157
158 always @(*) begin
159 case({ ALUOp ,funct7 ,funct3 })
160
161 12'b10_0000000_000: ALUcontrol_Out = 4'b0000;
162 12'b10_0100000_000: ALUcontrol_Out = 4'b0001;
163 12'b10_0000000_111: ALUcontrol_Out = 4'b0010;
164 12'b10_0000000_110: ALUcontrol_Out = 4'b0011;
165 default: ALUcontrol_Out = 4'b0000;
166
167 endcase
168 end
169 endmodule
170
171 module MUX2to1(
172 input [31:0] input0,
173 input [31:0] input1,
174 input select,
175 output [31:0] out
176 );
177
178 assign out = (select) ? input1 : input0;
179
180 endmodule
181
182 module Data_Memory(
183 input clk,
184 input rst,
185 input MemRead,
186 input MemWrite,
187 input [31:0] address,
188 input [31:0] write_data,
189 output [31:0] read_data
190 );

```

```

191
192 reg [31:0] D_Memory [0:255];
193 integer k;
194
195 assign read_data = (MemRead) ? D_Memory[address[9:2]] : 32'b0;
196
197 always @(posedge clk or posedge rst)
198 begin
199   if(rst)
200     begin
201       for(k=0;k<256;k=k+1)
202         D_Memory[k]=32'b0;
203     end
204   else if(MemWrite)
205     D_Memory[address[9:2]] = write_data;
206   end
207 endmodule
208
209 module Branch_Adder(
210   input [31:0] PC,
211   input [31:0] offset,
212   output [31:0] branch_target
213 );
214   assign branch_target = PC + offset;
215
216 endmodule
217
218
219 module RISCV_Top(input clk,input rst);
220
221   wire [31:0] pc_out_wire;
222   wire [31:0] pc_next_wire;
223   wire [31:0] pc_wire;
224   wire [31:0] decode_wire;
225   wire [31:0] read_data1;
226   wire [31:0] regtomux;
227   wire [31:0] WB_wire;
228   wire [31:0] branch_target;
229   wire [31:0] immgen_wire;
230   wire [31:0] muxtoAlu;
231   wire [31:0] read_data_wire;

```

```

232 wire [31:0] WB_data_wire;
233
234 wire RegWrite;
235 wire ALUSrc;
236 wire MemRead;
237 wire MemWrite;
238 wire MemToReg;
239 wire Branch;
240 wire Zero;
241
242 wire [1:0] ALUOp_wire;
243 wire [3:0] ALUcontrol_wire;
244
245 program_counter PC(clk,rst,pc_wire,pc_out_wire);
246
247 pc_adder PC_Adder(pc_out_wire,pc_next_wire);
248
249 pc_mux pc_mux_i(pc_next_wire,branch_target,Branch&Zero,pc_wire);
250
251 Instruction_Memory Instr_Mem(rst,clk,pc_out_wire,decode_wire);
252
253 Register_File Reg_File(clk,rst,RegWrite,decode_wire[19:15],
    decode_wire[24:20],decode_wire[11:7],WB_data_wire,read_data1,
    regtomux);
254
255 main_control_unit Control_Unit(decode_wire[6:0],RegWrite,MemRead,
    MemWrite,MemToReg,ALUSrc,Branch,ALUOp_wire);
256
257 ALU_Control ALU_Control_i(decode_wire[14:12],decode_wire[31:25],
    ALUOp_wire,ALUcontrol_wire);
258
259 ALU ALU_i(read_data1,muxtoAlu,ALUcontrol_wire,WB_wire,Zero);
260
261 immediate_generator Imm_Gen(decode_wire,immgen_wire);
262
263 MUX2to1 Imm_Mux(regtomux,immgen_wire,ALUSrc,muxtoAlu);
264
265 Data_Memory Data_Mem(clk,rst,MemRead,MemWrite,WB_wire,regtomux,
    read_data_wire);
266
267 MUX2to1 WB_Mux(WB_wire,read_data_wire,MemToReg,WB_data_wire);

```

```
268  
269 Branch_Adder Branch_Adder_i(pc_out_wire ,immgen_wire ,branch_target);  
270  
271 endmodule
```